Mathematical Modeling of Team Success in Baseball: A Sabermetric Approach

Kyla Igawa

School of Computing and Data Science

Wentworth Institute of Technology

August 10, 2025

Abstract

This report investigates the use of the Weibull distribution to predict baseball team win percentages using runs scored and runs allowed based on the Pythagorean won-loss theorem, while also creating a machine learning model to further explore predictive baseball statistics. The models discussed in the report consist of Weibull distribution modeling, least squares minimization, and machine learning approaches such as linear regression and softmax probability calculation. The results show that the Weibull-based model closely approximates observed win percentages from historical data, validating its effectiveness in sports analytics. Additionally, the machine learning model successfully estimates expected runs scored based on batting averages and starting pitcher ERA, providing probability estimates for team matchups. These findings suggest that combining traditional statistical models with machine learning techniques enhances predictive accuracy in baseball forecasting. While the Weibull distribution provides strong season-level win percentage estimates, the machine learning model is more suited for day-to-day game predictions, offering more specific and dynamic insights for strategic decision-making.

Contents

1 Introduction **4**

2 Background **5**

2.1 The Pythagorean Win–Loss Formula 5

2.2 Probability Basics: PDF and CDF 5

2.3 The Weibull Distribution 6

3 Derivation of the Pythagorean Formula using Weibull **7**

4 2004 American League Example **8**

4.1 Binning Strategy and the Role of β 8

4.2 Applying Least Squares to Fit Weibull Distributions 9

4.3 BFGS (Quasi-Newton) Algorithm 10

4.4 Results 13

5 Extensions: Simulation and Machine Learning **14**

5.1 Regression models 14

5.2 Linear Regression Model in Python 15

6 Conclusion **16**

7 References **17**

8 Appendix **18**

1 Introduction

The relationship between a baseball team's offensive and defensive performance and its overall success has long been of interest in sabermetrics. One of the most widely used tools for estimating team performance is the Pythagorean win–loss formula, originally introduced by Bill James, which models win percentage as a function of runs scored (RS) and runs allowed (RA). While elegant and effective, the original formulation assumes a fixed exponent and does not explicitly model the statistical distribution of runs.

In this work, a probabilistic framework is introduced by modeling RS and RA as independent random variables following the Weibull distribution. This approach provides a natural derivation of the Pythagorean win–loss formula and allows parameter tuning to improve predictive accuracy. Using historical season data, the Weibull distribution parameters are estimated using least squares minimization, and the BFGS quasi–Newton algorithm is employed to efficiently find optimal values.

This report extends beyond season-level prediction by exploring a game-by-game simulation model and incorporating regression-based machine learning techniques to forecast individual matchups.

These extensions demonstrate how statistical modeling and computational methods can complement one another, offering accurate season projections through the Weibull-based model, and more dynamic, short-term insights through machine learning.

The remainder of this report outlines the theoretical basis for the Weibull approach, details the parameter estimation procedure, presents empirical results on historical baseball data, and discusses the extension of these methods to simulation and predictive analytics.

2 Background

2.1 The Pythagorean Win–Loss Formula

The Pythagorean win–loss formula, introduced by Bill James, is a statistical model designed to estimate a baseball team's expected winning percentage based on the number of runs scored (RS) and runs allowed (RA). The standard form of the model is given by:

$$\text{Win\%} = \frac{\text{RS}^\gamma}{\text{RS}^\gamma + \text{RA}^\gamma} \tag{1}$$

where γ is an empirically determined exponent, originally set to 2 by James. The intuition is simple: a team's success is a balance between its scoring ability and its ability to prevent opponents from scoring, but the relationship is nonlinear due to the exponent γ. Later empirical studies found that $\gamma \approx 1.8$ often gives a better fit for Major League Baseball data.

2.2 Probability Basics: PDF and CDF

A key part of our approach is treating runs scored and runs allowed as *random variables* rather than fixed numbers. Two fundamental tools for describing random variables are the **probability density function** (PDF) and the **cumulative distribution function** (CDF).

For a continuous random variable X:

- The PDF, $f_X(x)$, describes the *relative likelihood* of observing a value near x.
- The CDF, $F_X(x)$, gives the probability that X takes on a value less than or equal to x:

$$F_X(x) = \int_0^x f_X(t)\,dt \tag{2}$$

- The PDF and CDF are related by differentiation:

$$f_X(x) = \frac{d}{dx} F_X(x) \tag{3}$$

The area under the entire PDF curve is always 1, representing the fact that the probability of *all possible outcomes* is 100%.

2.3 The Weibull Distribution

The Weibull distribution is a flexible family of continuous probability distributions characterized by three parameters:

- **Shape parameter** $\gamma > 0$: Controls the curvature of the distribution and tail behavior.
- **Scale parameter** $\alpha > 0$: Stretches or compresses the distribution along the horizontal axis.
- **Location parameter** $\beta = -0.5$: Shifts the distribution along the horizontal axis.

The PDF of the three-parameter Weibull distribution is:

$$f(x;\alpha,\beta,\gamma) \;=\; \begin{cases} \frac{\gamma}{\alpha}\left(\frac{x-\beta}{\alpha}\right)^{\gamma-1}\exp\left[-\left(\frac{x-\beta}{\alpha}\right)^{\gamma}\right], & x \geq \beta, \\ 0, & x < \beta \end{cases} \tag{4}$$

The CDF is:

$$F(x;\alpha,\beta,\gamma) \;=\; 1-\exp\left[-\left(\frac{x-\beta}{\alpha}\right)^{\gamma}\right], \quad x \geq \beta \tag{5}$$

Why the Weibull is a good choice. The Weibull distribution can take on a variety of shapes depending on γ:

- If $\gamma < 1$, the distribution has a heavy right tail.
- If $\gamma = 1$, it reduces to the exponential distribution.
- If $\gamma > 1$, it has a unimodal, bell-like shape.

This flexibility allows it to accurately model the asymmetric and skewed nature of baseball run distributions. The inclusion of the location parameter β (often set to -0.5 in this

application) accounts for the fact that runs are discrete and start at zero but are treated in a continuous model.

In our analysis, runs scored and runs allowed are modeled as *independent* Weibull random variables, making the probability a team wins:

$$P(\text{win}) \;=\; P(\text{RS} > \text{RA})$$

which can be expressed as a double integral over their respective PDFs—forming the starting point of the Weibull-based derivation of the Pythagorean formula.

We include an example Weibull PDF plot in the Appendix to visualize these shapes.

3 Derivation of the Pythagorean Formula using Weibull

The definition of a win in a game of baseball is that a team needs to score more runs than it allows. To compute this, we take $Prob(X > Y)$, where we model runs scored X and runs allowed Y as independent continuous random variables with densities f_X and f_Y.

$$Prob(X > Y) = \iint_{x \geq y} f_X(x)\, f_Y(y)\, dy\, dx$$

This integral calculation yields the closed form used in the paper:

$$Prob(X > Y) = \frac{(RS - \beta)^\gamma}{(RS - \beta)^\gamma + (RA - \beta)^\gamma},$$

where $(\alpha_{RS}, \beta, \gamma)$ and $(\alpha_{RA}, \beta, \gamma)$ are the Weibull parameters for runs scored and allowed respectively.

Remarks: This derivation assumes independence and treats integer run totals as continuous via the Weibull approximation; these choices make the integral tractable but are approximations to the discrete reality of baseball scores.

For the detailed step-by-step algebra, see the handwritten derivation in the Appendix.

4 2004 American League Example

The original paper analyzed the 14 teams of the American League from the 2004 season to test the reasonableness of the model's assumptions. The National League analysis was omitted and left as an exercise for the reader. For each team, the authors fit two separate three-parameter Weibull distributions: one to model runs scored per game, and one to model runs allowed per game. The location parameter was fixed at $\beta = -0.5$, leaving three free parameters: α_{RS}, α_{RA}, and γ.

In the paper, parameter estimation was carried out using two methods:

1. **Method of Least Squares:** Minimizing the sum of squared differences between the observed run distribution and the Weibull model for both runs scored and runs allowed.

2. **Method of Maximum Likelihood:** Selecting parameters that maximized the probability of observing the actual data given the Weibull model.

In my work, I replicated the analysis in Python, but only implemented the **method of least squares**. Using this approach, I determined the parameters α_{RS} and α_{RA} for each team while enforcing a common shape parameter γ. These fitted parameters were then used to predict each team's number of wins, losses, and winning percentage, which I compared to the actual 2004 season results to assess the accuracy of the model.

4.1 Binning Strategy and the Role of β

To adapt the Pythagorean Win-Loss model to sports beyond baseball, a binning strategy is introduced that partitions score data into intervals. This approach addresses several modeling challenges:

1. **Cross-sport applicability:** Different sports have varying scoring distributions. Binning allows the model to generalize across domains.

2. **Baseline adjustment:** It is more natural to measure scores relative to a baseline (e.g., average or minimum expected score), which motivates the introduction of a shift parameter β.

3. **Discrete vs. continuous tension:** While the model assumes a continuous score distribution, real-world data is discrete. Binning helps reconcile this mismatch by aggregating scores into intervals.

The binning intervals are defined as:

$$[-0.5, 0.5) \cup [0.5, 1.5) \cup [1.5, 2.5) \cup [2.5, 3.5) \cup \cdots \cup [11.5, \infty)$$

4.2 Applying Least Squares to Fit Weibull Distributions

The method of least squares is a standard technique in mathematical modeling used to find the best fit between a model and observed data. The goal is to minimize the total squared difference between observed values O_i and expected values E_i:

$$\text{Total Error} = \sum_i (O_i - E_i)^2$$

This principle ensures that the fitted model best captures the underlying distribution of outcomes across all bins.

We apply this method to fit Weibull cumulative distribution functions (CDFs) to the run distributions of each of the 14 American League teams from the 2004 season. Game outcomes are binned into integer intervals from 0 to 12 runs.

Let $RS_{\text{obs}}(k)$ and $RA_{\text{obs}}(k)$ denote the observed frequencies of runs scored and runs allowed in bin k, respectively. We model the expected frequencies using the Weibull CDF, denoted $A(\alpha, \beta, \gamma, k)$, where:

- α is the scale parameter
- $\beta = -0.5$ is a fixed location shift
- γ is the shape parameter
- k is the bin index

The expected frequency in bin k is given by:

$$E(k) = \#\text{Games} \cdot A(\alpha, -0.5, \gamma, k)$$

To fit the Weibull parameters, we minimize the total squared error between observed and expected frequencies for both runs scored (RS) and runs allowed (RA):

$$\sum_{k=1}^{\#\text{Bins}} (RS_{\text{obs}}(k) - \#\text{Games} \cdot A(\alpha_{RS}, -0.5, \gamma, k))^2 + \sum_{k=1}^{\#\text{Bins}} (RA_{\text{obs}}(k) - \#\text{Games} \cdot A(\alpha_{RA}, -0.5, \gamma, k))^2$$

This objective function is minimized using the BFGS algorithm, a quasi-Newton method that approximates the Hessian to efficiently converge to optimal parameter values.

4.3 BFGS (Quasi-Newton) Algorithm

The goal of least squares optimization is to minimize the total squared error between observed data and model predictions:

$$\text{Total Error} = \sum_i (O_i - E_i)^2$$

To achieve this, we use the BFGS algorithm (Broyden-Fletcher-Goldfarb-Shanno), a quasi-Newton method that iteratively searches for optimal parameters by approximating curvature and leveraging gradient information.

Step 1: Compute the Function Value and Gradient

(a) Evaluate the objective function at the current point:

$$f(x_k)$$

(b) Compute the gradient:

$$g_k = \nabla f(x_k)$$

(c) **Gradient meaning**: A vector of partial derivatives pointing in the direction of steepest ascent. It acts like an arrow indicating how the surface rises most rapidly.

Step 2: Compute the Search Direction

(a) Use the inverse Hessian approximation to compute the search direction:

$$d_k = -H_k^{-1} \nabla f(x_k)$$

(b) **Quasi-Newton step**: This direction moves opposite the gradient but is scaled based on curvature, allowing more efficient descent.

(c) **Hessian meaning**: A matrix of second derivatives that describes local curvature. It tells us how the gradient is changing.

(d) Curvature interpretation:

- Positive eigenvalues $\rightarrow$ bowl shape $\rightarrow$ local minimum
- Negative eigenvalues $\rightarrow$ dome shape $\rightarrow$ local maximum
- Mixed signs $\rightarrow$ saddle point

Step 3: Choose Step Size and Update Parameters

(a) Update the parameters using the chosen step size α_k:

$$x_{k+1} = x_k + \alpha_k \cdot d_k$$

(b) **Step size selection**: Use Wolfe conditions to ensure the step is effective.

(c) **Armijo Rule (Sufficient Decrease)**:

- Ensures the step reduces the function value enough.
- Compares the new function value to a linear approximation based on the current gradient.

(d) **Curvature Condition**:

- Ensures the gradient at the new point is properly aligned with the search direction.
- Prevents shallow moves or sliding along the edge of the bowl.

Step 4: Update the Hessian Approximation

(a) Compute:

$$s_k = x_{k+1} - x_k$$
$$y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$$

(b) Update the Hessian approximation:

$$H_{k+1} = H_k - \frac{H_k s_k s_k^T H_k}{s_k^T H_k s_k} + \frac{y_k y_k^T}{y_k^T s_k}$$

(c) **Interpretation**:

- The first term removes outdated curvature along the step direction.

- The second term adds curvature in the direction of gradient change.
- This update satisfies the **secant equation**: $H_{k+1}s_k = y_k$

Step 5: Check for Convergence

(a) The algorithm repeats until one of the following is satisfied:

- Step size α_k becomes very small
- Change in function value is negligible
- Gradient norm is close to zero (flat slope)

4.4 Results

Using least squares minimization, I replicated the fitting procedure in Python (code included in the appendix). For each of the 14 American League teams from the 2004 season, I computed the optimal parameters for their respective Weibull distributions.

With these fitted parameters, I applied the updated Pythagorean Win-Loss Theorem to estimate each team's win percentage. The results include:

- Observed wins
- Predicted wins
- Observed win percentage
- Predicted win percentage
- Games difference
- Estimated shape parameter γ

A summary table of these results, along with a visual representation of the least squares output, is provided in the appendix. These findings illustrate the model's ability to closely approximate team performance and validate the predictive utility of the Weibull-based Pythagorean framework.

5 Extensions: Simulation and Machine Learning

Following the replication of win percentage results using the updated Pythagorean Win-Loss Theorem, I explored a more granular approach to modeling baseball outcomes on a game-by-game basis. The goal was to develop a predictive framework that could simulate individual matchups using team-level offensive metrics and starting pitcher statistics.

To achieve this, I designed a Python-based simulation that allows users to input key offensive statistics (e.g., avg, obp, slg) for two competing teams, along with the opposing starting pitchers' ERA. The model then estimates the expected number of runs scored by each team and predicts the likely winner of the matchup.

This required two core components:

- A probabilistic simulation engine to emulate the dynamics of a baseball game using statistical inputs.
- A regression-based predictive model trained on historical data to generate realistic scoring outcomes and win probabilities.

By integrating simulation with machine learning, this extension moves beyond season-level aggregates and enables more nuanced, matchup-specific analysis. The full implementation and code are provided in the appendix.

5.1 Regression models

To predict runs scored in simulated baseball games, I evaluated three regression techniques using a dataset split into 80% training and 20% testing. The objective was to construct a model that could reliably estimate scoring outcomes based on team-level offensive statistics and opposing pitcher ERA.

The following approaches were considered:

Linear Regression (Baseline): Linear regression served as the baseline model due to its simplicity and high interpretability. It assumes a linear relationship between input features and the target variable, making it a useful benchmark for evaluating more complex models.

K-Nearest-Neighbors (KNN): KNN is a non-parametric method that predicts outcomes by averaging the values of the nearest training examples. I tuned the number of neighbors k to minimize test error while avoiding overfitting. The graph titled *KNN Performance vs. Number of Neighbors*, included in the appendix, illustrates this tuning process.

Polynomial Regression: To capture nonlinear interactions between batting and pitching metrics, I implemented polynomial regression with varying degrees and interaction terms. This feature expansion improved model flexibility and fit, as shown in the graph *Adjusted Polynomial Regression*, also included in the appendix.

Mean Squared Error Evaluation: Each model was evaluated using mean squared error (MSE) on the test set. This metric provided a quantitative measure of prediction accuracy and helped assess the trade-offs between model complexity and generalization. A summary of the MSE results is presented in the appendix.

5.2 Linear Regression Model in Python

To extend the analysis beyond season-level aggregates, I developed a simulation-based framework for forecasting individual baseball games. This approach is particularly useful for applications in sports analytics and predictive modeling.

The core of the system is a custom baseball simulation in Python, approximately 50 lines in length, which models games inning-by-inning. It uses key offensive statistics—batting average (AVG), on-base percentage (OBP), and slugging percentage (SLG)—alongside the opposing team's earned run average (ERA) to probabilistically simulate scoring outcomes.

To model the relationship between these inputs and the number of runs scored, I applied linear regression to a polynomial-expanded feature set. Specifically, I used a second-degree polynomial expansion to capture nonlinear interactions between predictors while maintaining the interpretability of a linear model. The regression equation takes the form:

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

where y represents the predicted number of runs, x_i are the expanded features, and ϵ is the error term. The model minimizes the sum of squared errors between predicted and observed runs.

This framework demonstrated strong predictive performance and remains adaptable for future enhancements. Additional variables such as bullpen strength, home-field advantage, and weather conditions can be incorporated to further refine the simulation and improve accuracy.

An image of the input/output will be included in the appendix as well as a image of the script in full.

6 Conclusion

This project demonstrates how mathematical modeling and machine learning can be effectively combined to analyze and predict baseball team performance. By deriving the Pythagorean win–loss formula from first principles using the Weibull distribution, I provided a probabilistic foundation for estimating season-level win percentages. Least squares minimization and BFGS optimization enabled accurate parameter estimation, validating the model against historical data from the 2004 American League.

To extend the analysis beyond aggregate season outcomes, I developed a simulation-based framework for forecasting individual games. This included a custom Python simulator and regression models—most notably linear regression with polynomial feature expansion—to

predict runs scored based on team offensive metrics and opposing pitcher ERA. These models offer interpretable, data-driven insights and can be adapted to incorporate additional variables for enhanced realism.

Together, these approaches highlight the power of combining traditional statistical theory with modern computational techniques. The result is a flexible, multi-scale modeling framework that supports both strategic season planning and tactical game-level decision-making in baseball analytics.

7 References

- Miller, S. J. (2007). *A Derivation of the Pythagorean Won-Loss Formula in Baseball.* Retrieved from `https://web.williams.edu/Mathematics/sjmiller/public_html/399/handouts/PythagWonLoss_Paper.pdf`

- Honorio, J. (2016). *The BFGS Method.* Retrieved from `https://www.cs.purdue.edu/homes/jhonorio/16spring-cs52000-quasinewton.pdf`

- Wikipedia contributors. (2025). *Least Squares.* Retrieved from `https://en.wikipedia.org/wiki/Least_squares`

- Wikipedia contributors. (2025). *Probability Density Function.* Retrieved from `https://en.wikipedia.org/wiki/Probability_density_function`

- Baseball Almanac. (2025). *The History of Major League Baseball.* Retrieved from `https://www.baseball-almanac.com/`

- *An Introduction to Statistical Learning: with Applications in Python.* Springer. Retrieved from `https://www.statlearning.com/`

8 Appendix

The appendix includes, python code snippets as well as extra graphs and tables referenced in the report.

```python
import pandas as pd
import numpy as np
from scipy.optimize import minimize
from scipy.special import gamma as gamma_func
import matplotlib.pyplot as plt

#reads in excel sheet, assigns the columns names, teams for the first, RS_{i} for i through 26 including 0, and RA_{i} f
def load_data(file_path):  1 usage
    df = pd.read_excel(file_path, engine='openpyxl')
    df.columns = ['Team'] + [f'RS_{i}' for i in range(27)] + [f'RA_{i}' for i in range(27)]
    return df


#manually inputted wins from almanac page, returns a dictionary of wins per team and win percentage
def observed_wins():  1 usage
    observed = {
        'Boston Red Sox': 98,
        'New York Yankees': 101,
        'Baltimore Orioles': 78,
        'Tampa Bay Devil Rays': 70,
        'Toronto Blue Jays': 67,
        'Minnesota Twins': 92,
        'Chicago White Sox': 83,
        'Cleveland Indians': 80,
        'Detroit Tigers': 72,
        'Kansas City Royals': 58,
        'Los Angeles Angels': 92,
        'Oakland Athletics': 91,
        'Texas Rangers': 89,
        'Seattle Mariners': 63
    }
    total_games = 162
    return {team: (wins, round(wins / total_games, 3)) for team, wins in observed.items()}

# weibull area under the bin
def weibull_area(alpha, beta, gamma, bin_start, bin_end):  2 usages
    return np.exp(-((bin_start - beta) / alpha) ** gamma) - np.exp(-((bin_end - beta) / alpha) ** gamma)

#found by using the cdf formula (basicaly saying for each bin, what is the probability of this team scoring k runs (midp
# fit weibull and calculate predicted win percentage
def predicted_wins(df):  1 usage
    total_games = 162
```

Figure 1: Least squares code (part 1)

```python
    results = {}

    for _, row in df.iterrows():
        team = row['Team']
        rs_bins = row[[f'RS_{i}' for i in range(27)]].values
        ra_bins = row[[f'RA_{i}' for i in range(27)]].values

        #computes the error, equation given from the footnote (3.1) on bottom of page 5
        def error(params):
            alpha_rs, alpha_ra, gamma = params
            rs_error = sum((rs_bins[k] - total_games * weibull_area(alpha_rs, -0.5, gamma, k-0.5, k + .5)) ** 2 for k in
            ra_error = sum((ra_bins[k] - total_games * weibull_area(alpha_ra, -0.5, gamma, k-0.5, k + .5)) ** 2 for k in
            return rs_error + ra_error

        result = minimize(error, x0=[5, 5, 1.8], bounds=[(0.1, None), (0.1, None), (0.1, None)])
        print(result)


        alpha_rs, alpha_ra, gamma = result.x
        #5, 5, 1.8 are initial guesses for alpha of RS and RA and gamma
        rs_mean = alpha_rs * gamma_func(1 + 1 / gamma) + 0.5
        ra_mean = alpha_ra * gamma_func(1 + 1 / gamma) + 0.5
        #calculates mean from given equation in theorem 2.2

        pred_perc = (rs_mean - 0.5) ** gamma / ((rs_mean - 0.5) ** gamma + (ra_mean-0.5) ** gamma)
        pred_wins = pred_perc * total_games
        #this is the pythagorean win% formula
        results[team] = {
            'Pred Wins': pred_wins,
            'PredPerc': pred_perc,
            'Gamma': gamma
        }
    return results


def main():  1 usage
    file_path = 'C:/Users/15623/OneDrive/Documents/team_run_distributions_2004.xlsx'
    df = load_data(file_path)
```

Figure 2: Least squares code (part 2)

```python
    obs_data = observed_wins()
    pred_data = predicted_wins(df)

    summary = []
    for team in obs_data:
        obs_wins, obs_perc = obs_data[team]
        pred_wins = pred_data[team]['Pred Wins']
        pred_perc = pred_data[team]['PredPerc']
        gamma = pred_data[team]['Gamma']
        games_diff = obs_wins-pred_wins

        summary.append([team, obs_wins, pred_wins, obs_perc, pred_perc, games_diff, gamma])

    summary_df = pd.DataFrame(summary, columns=['Team', 'Obs Wins', 'Pred Wins', 'ObsPerc', 'PredPerc', 'GamesDiff', 'Ga
    pd.set_option('display.max_columns', None)
    pd.set_option('display.float_format', '{:.3f}'.format)

    print(summary_df)

if __name__ == "__main__":
    main()
```

Figure 3: Least squares code (part 3)

Team	Obs Wins	Pred Wins	ObsPerc	PredPerc	GamesDiff	Gamma
Boston Red Sox	98	94.396	0.605	0.583	3.604	1.797
New York Yankees	101	92.243	0.623	0.569	8.757	1.769
Baltimore Orioles	78	84.027	0.481	0.519	-6.027	1.630
Tampa Bay Devil Rays	70	72.334	0.432	0.447	-2.334	1.812
Toronto Blue Jays	67	64.705	0.414	0.399	2.295	2.000
Minnesota Twins	92	91.647	0.568	0.566	0.353	1.801
Chicago White Sox	83	81.569	0.512	0.504	1.431	1.707
Cleveland Indians	80	80.184	0.494	0.495	-0.184	1.807
Detroit Tigers	72	78.117	0.444	0.482	-6.117	1.757
Kansas City Royals	58	61.808	0.358	0.382	-3.808	1.800
Los Angeles Angels	92	92.251	0.568	0.569	-0.251	1.683
Oakland Athletics	91	79.615	0.562	0.491	11.385	1.787
Texas Rangers	89	85.615	0.549	0.528	3.385	1.827
Seattle Mariners	63	72.785	0.389	0.449	-9.785	1.762

Figure 4: Least squares fitting results

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt


def simulate_game(avg, obp, slg, era):  1 usage
    runs = 0
    baseline_era = 4.5
    era_diff = era - baseline_era
    adjustment = era_diff * 0.005
    avg += adjustment
    obp += adjustment
    for innings in range(9):
        outs = 0
        bases = [0, 0, 0]
        while outs < 3:
            outcome = np.random.rand()
            walk_prob = max(0, obp - avg)
            hit_prob = avg
            if outcome < walk_prob:
                if all(bases): runs += 1
                bases = [1] + bases[:2]
            elif outcome < walk_prob + hit_prob:
                tb_per_hit = slg / avg
                base_probs = [0.6, 0.25, 0.05, 0.1]
                adj = tb_per_hit - 1.0
                base_probs = [
                    base_probs[0] - 0.1 * adj,
                    base_probs[1] + 0.05 * adj,
                    base_probs[2] + 0.02 * adj,
                    base_probs[3] + 0.03 * adj,
                ]
                total = sum(base_probs)
                base_probs = [x / total for x in base_probs]
                hit_type = np.random.choice( a: ['1B', '2B', '3B', 'HR'], p=base_probs)
                if hit_type == 'HR':
```

Figure 5: Model testing code (part 1)

```
                    runs += 1 + sum(bases)
                    bases = [0, 0, 0]
                elif hit_type == '3B':
                    runs += sum(bases)
                    bases = [0, 0, 1]
                elif hit_type == '2B':
                    runs += bases[2] + bases[1]
                    bases = [0, 1, 0]
                else:
                    runs += bases[2]
                    bases = [1] + bases[:2]
            else:
                outs += 1
    return max(0, runs)


def generate_data(n=1000):  1 usage
    np.random.seed(42)
    avgs = np.random.normal( loc: 0.270,  scale: 0.015, n)
    obps = avgs + np.random.normal( loc: 0.055,  scale: 0.015, n)
    slgs = np.random.normal( loc: 0.420,  scale: 0.040, n)
    eras = np.random.normal( loc: 4.50,  scale: 0.55, n)
    runs = [simulate_game(avgs[i], obps[i], slgs[i], eras[i]) for i in range(n)]
    X = np.column_stack((avgs, obps, slgs, eras))
    y = np.array(runs)
    return X, y


def evaluate_models():  1 usage
    X, y = generate_data()

    #Split data (80% train, 20% test)
    X_train, X_test, y_train, y_test = train_test_split( *arrays: X, y, test_size=0.2, random_state=42)

    errors = {}

    linreg = LinearRegression()
    linreg.fit(X_train, y_train)
    y_pred = linreg.predict(X_test)
    errors["Linear"] = mean_squared_error(y_test, y_pred)
```

Figure 6: Model testing code (part 2)

```python
#Polynomial Regression (degree=2)
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)
polyreg = LinearRegression()
polyreg.fit(X_train_poly, y_train)
y_pred_poly = polyreg.predict(X_test_poly)
errors["Poly (deg=2)"] = mean_squared_error(y_test, y_pred_poly)
##see which terms did better
#on training data it will look better, when we add interaction terms, see how well it does on the held out data,

# Fit polynomial regression
poly = PolynomialFeatures(degree=2, include_bias=False)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

polyreg = LinearRegression()
polyreg.fit(X_train_poly, y_train)
y_pred_poly = polyreg.predict(X_test_poly)

errors["Poly (deg=2)"] = mean_squared_error(y_test, y_pred_poly)

#Plot feature importance for polynomial terms
input_feature_names = ['avgs', 'obps', 'slgs', 'eras']
poly_feature_names = poly.get_feature_names_out(input_feature_names)
coeffs = polyreg.coef_
abs_coeffs = np.abs(coeffs)

# Sort by absolute value
sorted_idx = np.argsort(abs_coeffs)[::-1]
sorted_names = poly_feature_names[sorted_idx]
sorted_values = abs_coeffs[sorted_idx]

plt.figure(figsize=(10, 8))
plt.barh(sorted_names, sorted_values, color='skyblue')
plt.xlabel('Coefficient Magnitude (Importance)')
plt.title('📐 Polynomial Feature Importances (degree=2)')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()
```

Figure 7: Model testing code (part 3)

```python
    #knn
    k_values = range(1, 101, 2)
    mse_scores = []

    for k in k_values:
        knn = KNeighborsRegressor(n_neighbors=k)
        knn.fit(X_train, y_train)
        y_pred = knn.predict(X_test)
        mse_scores.append(mean_squared_error(y_test, y_pred))

    plt.plot( *args: k_values, mse_scores)
    plt.xlabel("Number of Neighbors (k)")
    plt.ylabel("Test MSE")
    plt.title("KNN Performance vs. Number of Neighbors")
    plt.grid(True)
    plt.show()

    knn = KNeighborsRegressor(n_neighbors=35)
    knn.fit(X_train, y_train)
    y_pred_knn = knn.predict(X_test)
    errors["KNN (k=25)"] = mean_squared_error(y_test, y_pred_knn)


    print("\nMean Squared Errors (lower is better):")
    for name, mse in errors.items():
        print(f"{name}: {mse:.3f}")

    plt.figure(figsize=(10, 5))
    plt.bar(errors.keys(), errors.values(), color='skyblue')
    plt.ylabel("Test MSE")
    plt.title("Model Comparison (Test Error)")
    plt.xticks(rotation=20)
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    evaluate_models()
```

Figure 8: Model testing code (part 4)

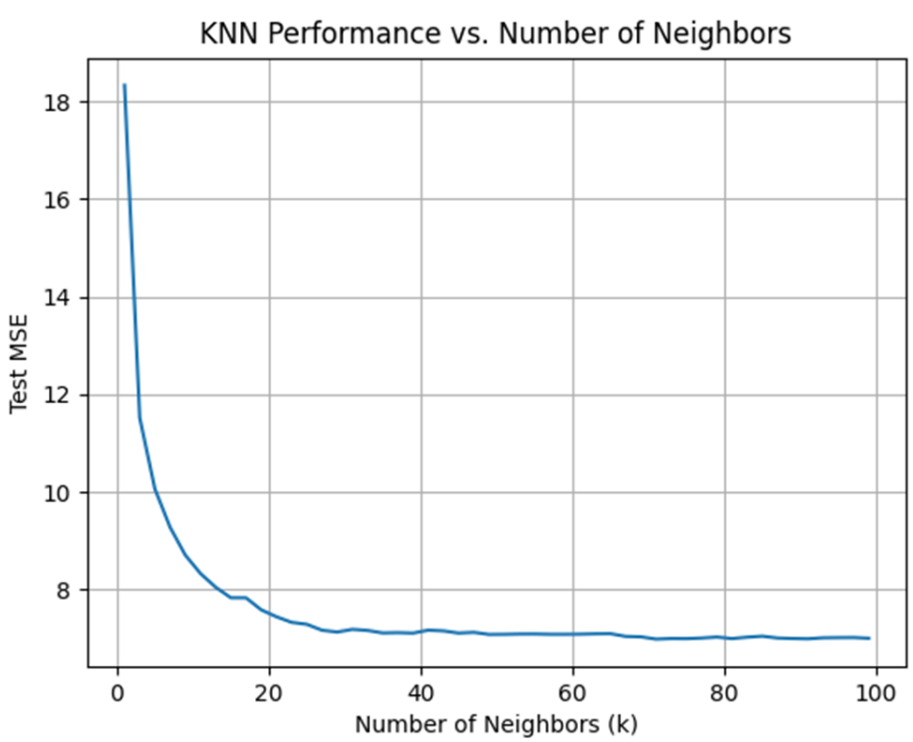


Figure 9: KNN model MSE across different k-values

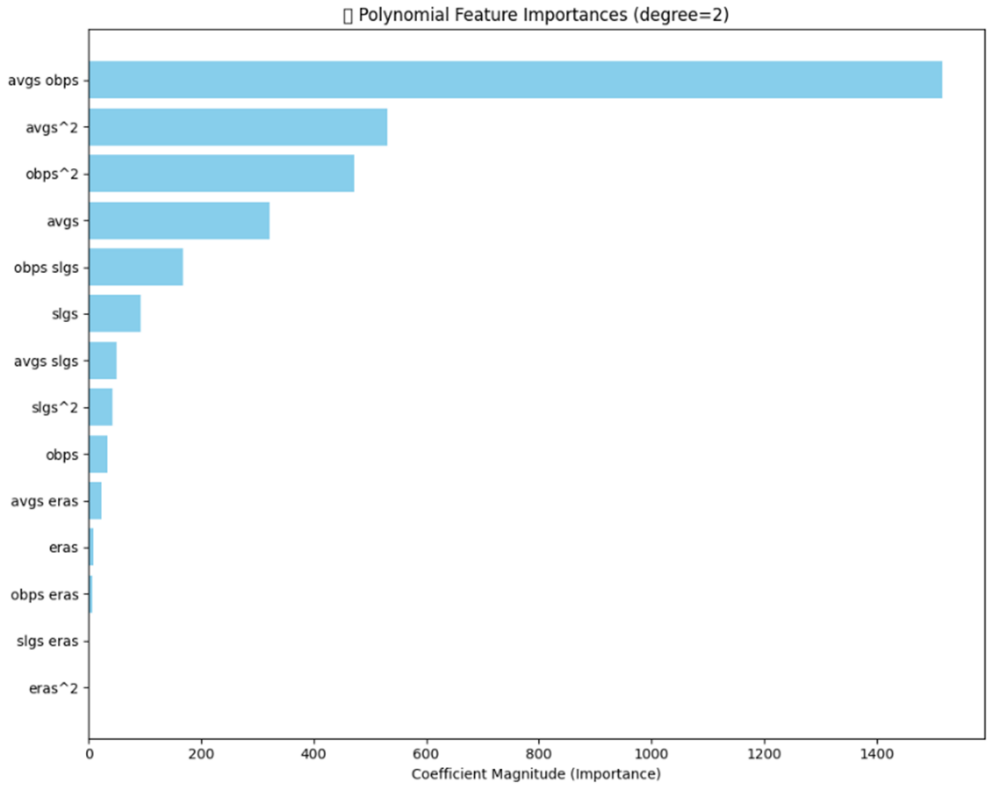


Figure 10: Polynomial feature importance

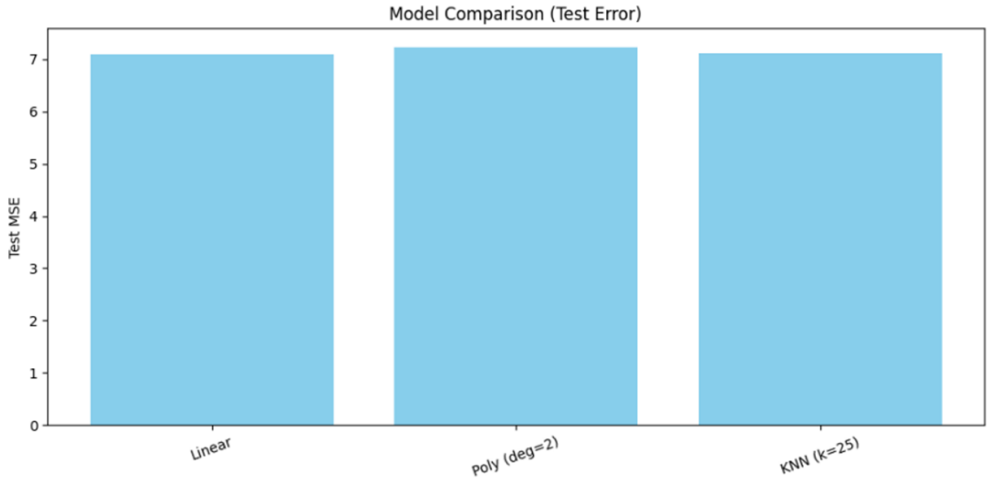


Figure 11: Comparison of MSE across models

```python
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures

import numpy as np

def simulate_game(avg, obp, slg, era):  1 usage
    runs = 0
    baseline_era = 4.5 #should be changed to current year (using 2004)
    era_diff = era - baseline_era  # negative if pitcher is better than average
    adjustment = era_diff * 0.005  # adjust avg and obp by this per 1.00 difference in ERA
    #rough calculations ^, more or less runs per 9 innings, on average how many hits does it take to score a run,how muc
    # Adjust AVG and OBP based on ERA difference
    avg = avg + adjustment
    obp = obp + adjustment #modified adjustment with walks !!future work!!

    for inning in range(9):
        outs = 0
        bases = [0, 0, 0]  # first, second, third
        while outs < 3:
            outcome = np.random.rand()
            walk_prob = max(0, obp - avg)
            hit_prob = avg

            if outcome < walk_prob:
                # Walk
                if bases[0] and bases[1] and bases[2]:
                    runs += 1
                bases = [1] + bases[:2]
            elif outcome < walk_prob + hit_prob:
                # Estimate expected bases per hit
                tb_per_hit = slg / avg
                base_probs = [0.6, 0.25, 0.05, 0.1]  # Default weights: [1B, 2B, 3B, HR]

                # Slightly shift probabilities based on tb_per_hit
                adj = tb_per_hit - 1.0  # offset from single
                base_probs = [
                    base_probs[0] - 0.1 * adj,
                    base_probs[1] + 0.05 * adj,
                    base_probs[2] + 0.02 * adj,
                    base_probs[3] + 0.03 * adj,
                ]
```

Figure 12: Linear regression code (part 1)

```python
                # Normalize to ensure sum to 1
                total = sum(base_probs)
                base_probs = [x / total for x in base_probs]
                #divides each element of the list by the total sum,
                #which converts each one into a proportion of the whole
                #all probs add up to 1

                hit_type = np.random.choice( a: ['1B', '2B', '3B', 'HR'], p=base_probs)

                if hit_type == 'HR':
                    runs += 1 + sum(bases)
                    bases = [0, 0, 0]
                elif hit_type == '3B':
                    runs += sum(bases)
                    bases = [0, 0, 1]
                elif hit_type == '2B':
                    runs += bases[2] + bases[1]
                    bases = [0, 1, 0]
                else:  # single
                    runs += bases[2]
                    bases = [1] + bases[:2]
            else:
                outs += 1

    return max(0, runs)

#does the results fit the weibull distribution? do model for 1000s
#maybe bin the results and recreate least squares
#maybe weibull was a good choice maybe a different distribution is better
#statistical tests (hypothesis) check if a data set fits a distribution

# Generate synthetic data using the simulation
def generate_synthetic_data(num_samples=1000): #6  1 usage
    np.random.seed(42)
    avgs = np.random.normal( loc: 0.270,  scale: 0.015, num_samples)
    obps = avgs + np.random.normal( loc: 0.055,  scale: 0.015, num_samples)
    slgs = np.random.normal( loc: 0.420,  scale: 0.040, num_samples)
    eras = np.random.normal( loc: 4.50,  scale: 0.55, num_samples)
```

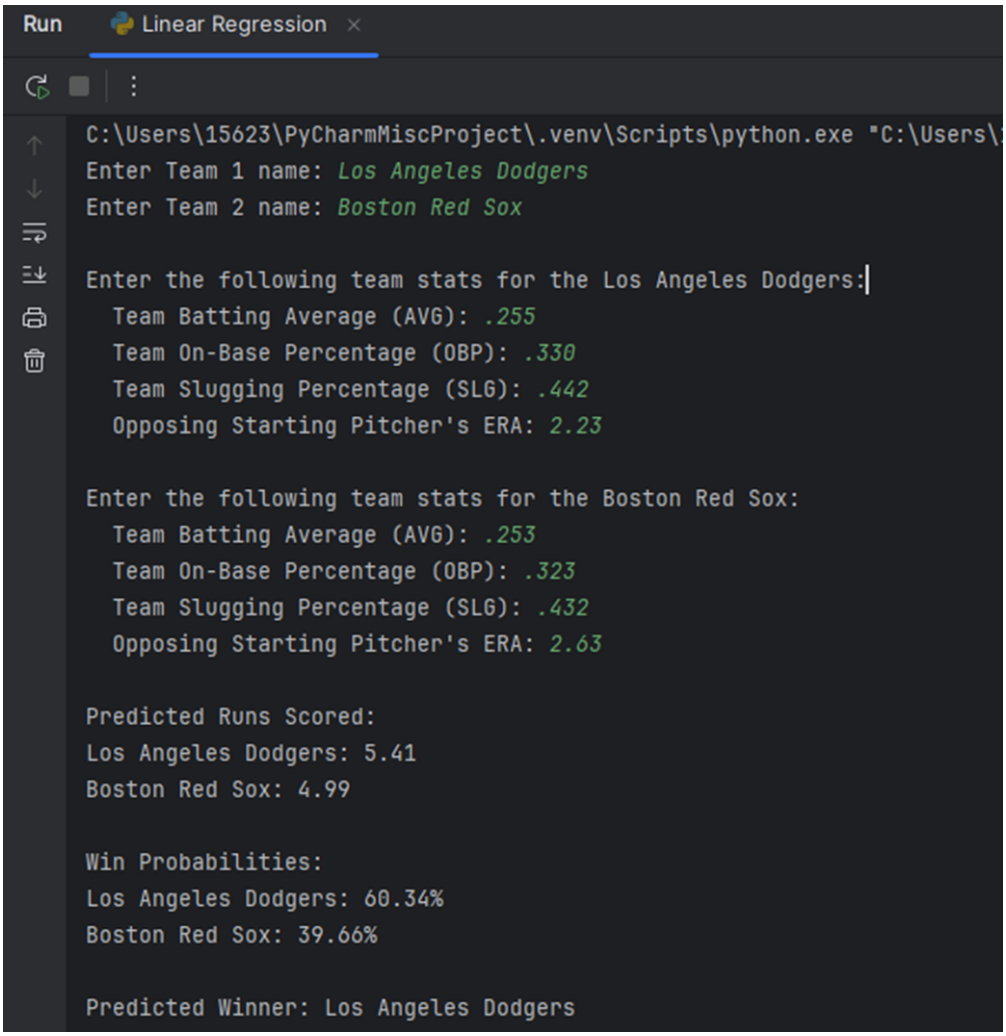
Figure 13: Linear regression code (part 2)

```python
    runs_scored = []
    for i in range(num_samples):
        runs = simulate_game(avgs[i], obps[i], slgs[i], eras[i])
        runs_scored.append(runs)

    return avgs, obps, slgs, eras, np.array(runs_scored)

# Train polynomial regression model
def train_model(): #4  1 usage
    avgs, obps, slgs, eras, runs_scored = generate_synthetic_data() #5
    X = np.column_stack((avgs, obps, slgs, eras))
    #tell which of the four is most important (p-value)
    poly = PolynomialFeatures(degree=2, include_bias=False)
    #Polynomial Features helps expand the feature set by adding interaction and polynomial terms
    # original: (x1,x2), interaction: (x1*x2), polynomial: (x1^2, x2^2)
    X_poly = poly.fit_transform(X)
    # a new set that has (avg, obps, slgs, eras, and expands with interaction and polynomial terms)
    model = LinearRegression()
    #^ this model will learn how to predict runs scored based on the expanded features
    model.fit(X_poly, runs_scored)
    #^trains the model using: X_poly (expanded features from simulated games)
    #and runs_scored (the actual simulated run totals)
    return model, poly

# Get user input for team stats
def get_team_input(team_name):#10  2 usages
    print(f"\nEnter the following team stats for the {team_name}:")
    avg = float(input("  Team Batting Average (AVG): "))
    obp = float(input("  Team On-Base Percentage (OBP): "))
    slg = float(input("  Team Slugging Percentage (SLG): "))
    era = float(input("  Opposing Starting Pitcher's ERA: "))
    return avg, obp, slg, era

# Predict runs using the trained model

def predict_runs(model, poly, avg, obp, slg, opp_era):#12  2 usages
    features = np.array([[avg, obp, slg, opp_era]])
    features_poly = poly.transform(features)
```

Figure 14: Linear regression code (part 3)

```python
    #takes team stats and expands them into a richer set of features (stat, stat squared, ie. obp*slg)
    prediction = model.predict(features_poly)[0]
    return max(0,prediction)

# Main function for team matchup
def team_matchup(): #2  1 usage
    model, poly = train_model() #3

    team1 = input("Enter Team 1 name: ").strip()
    team2 = input("Enter Team 2 name: ").strip()

    avg1, obp1, slg1, era2 = get_team_input(team1) #8
    avg2, obp2, slg2, era1 = get_team_input(team2)

    runs1 = predict_runs(model, poly, avg1, obp1, slg1, era2) #11
    runs2 = predict_runs(model, poly, avg2, obp2, slg2, era1)

    expscores = np.exp([runs1, runs2])
    #exponentials used to ensure all values are positive and dividing ensures prob add up to 1
    probs = expscores/np.sum(expscores)

    print(f"\nPredicted Runs Scored:")
    print(f"{team1}: {runs1:.2f}")
    print(f"{team2}: {runs2:.2f}")

    print(f"\nWin Probabilities:")
    print(f"{team1}: {probs[0]*100:.2f}%")
    print(f"{team2}: {probs[1] * 100:.2f}%")
    winner = team1 if runs1 > runs2 else team2
    print(f"\nPredicted Winner: {winner}")
#hold out some of my data for testing, when i am trying to decide using polynomial, compare which one is better
#which one does better on test out data. test error vs different models i did


if __name__ == "__main__":
    team_matchup() #1
```

Figure 15: Linear regression code (part 4)



```
Run    Linear Regression
C:\Users\15623\PyCharmMiscProject\.venv\Scripts\python.exe "C:\Users\
Enter Team 1 name: Los Angeles Dodgers
Enter Team 2 name: Boston Red Sox

Enter the following team stats for the Los Angeles Dodgers:
  Team Batting Average (AVG): .255
  Team On-Base Percentage (OBP): .330
  Team Slugging Percentage (SLG): .442
  Opposing Starting Pitcher's ERA: 2.23

Enter the following team stats for the Boston Red Sox:
  Team Batting Average (AVG): .253
  Team On-Base Percentage (OBP): .323
  Team Slugging Percentage (SLG): .432
  Opposing Starting Pitcher's ERA: 2.63

Predicted Runs Scored:
Los Angeles Dodgers: 5.41
Boston Red Sox: 4.99

Win Probabilities:
Los Angeles Dodgers: 60.34%
Boston Red Sox: 39.66%

Predicted Winner: Los Angeles Dodgers
```

Figure 16: Linear regression model results

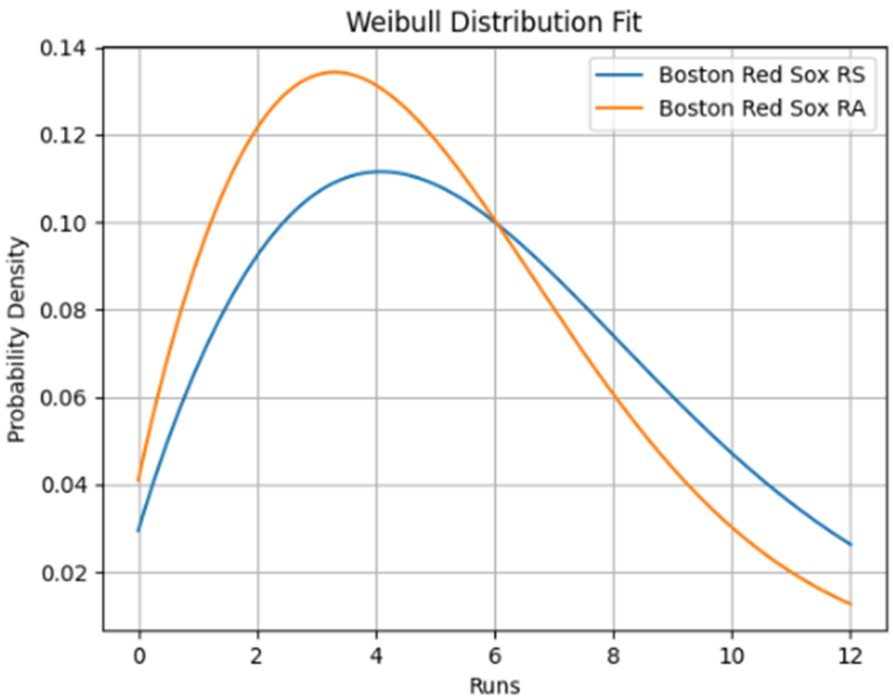


Figure 17: Weibull distribution fit

Solve Theorem 2.2 (Pythag Won-Loss Formula)

$$\text{Prob}(X > Y) = \int_{x=\beta}^{\infty} \int_{y=\beta}^{x} f(x; d_{PS}, \beta, \gamma) f(y; d_{PA}, \beta, \gamma)\, dy\, dx$$

plug in equations

$$= \int_{x=\beta}^{\infty} \int_{y=\beta}^{x} \frac{\gamma}{d_{PS}} \left(\frac{x-\beta}{d_{PS}}\right)^{\gamma-1} e^{-((x-\beta)/d_{PS})^{\gamma}} \cdot \frac{\gamma}{d_{PA}} \left(\frac{y-\beta}{d_{PA}}\right)^{\gamma-1} e^{-((y-\beta)/d_{PA})^{\gamma}}\, dy\, dx$$

the next step involves separating the double integral and changing the bounds. it looks like this

$y = y - \beta \quad x = x - \beta$

this centers the weibull at 0 instead of β

$$= \int_{x=0}^{\infty} \frac{\gamma}{d_{PS}} \left(\frac{x}{d_{PS}}\right)^{\gamma-1} e^{-(x/d_{PS})^{\gamma}} \left[\int_{y=0}^{x} \frac{\gamma}{d_{PA}} \left(\frac{y}{d_{PA}}\right)^{\gamma-1} e^{-(y/d_{PA})^{\gamma}}\, dy\right] dx$$

inside the brackets: $\int_0^x f_Y(y)\,dy = \text{Prob}(Y \le x) = F_Y(x)$

so: $\text{Prob}(X > Y) = \int_0^x f_X(x) \cdot [\quad F_Y(x)]\, dx$

this says: for each possible run total x, the team wins if the opponent scores less than x

$$= \int_0^{\infty} \frac{\gamma}{d_{PS}} \left(\frac{x}{d_{PS}}\right)^{\gamma-1} \cdot e^{-(x/d_{PS})^{\gamma}} \cdot \left[1 - e^{-(x/d_{PA})^{\gamma}}\right] dx$$ ← negative 1 is from $\int_0^x f_X(x)\,dx - \int_0^{\infty} F_Y(x) f_X(x)\,dx$

use exponential identity: $e^{-a} \cdot e^{-b} = e^{-(a+b)}$

$$= 1 - \int_0^{\infty} \frac{\gamma}{d_{PS}} \left(\frac{x}{d_{PS}}\right)^{\gamma} \cdot e^{-[(x/d_{PS})^{\gamma} + (x/d_{PA})^{\gamma}]}\, dx \qquad = 1 - \int_0^{\infty} f_X(x) F_Y(x)\, dx$$

now we combine exponential terms:

$$= 1 - \int_0^{\infty} \frac{\gamma}{d_{PS}} \left(\frac{x}{d_{PS}}\right)^{\gamma} \cdot e^{-(x^{\gamma})(1/d_{PS}^{\gamma} + 1/d_{PA}^{\gamma})}\, dx$$

let $\frac{1}{\alpha^{\gamma}} = \frac{1}{d_{PS}^{\gamma}} + \frac{1}{d_{PA}^{\gamma}}$ so $\quad = 1 - \int_0^{\infty} \frac{\gamma}{d_{PS}} \left(\frac{x}{d_{PS}}\right)^{\gamma-1} e^{-(x/\alpha)^{\gamma}}\, dx$

Figure 18: Pythagorean formula derivation (part 1)

NOTE: $\alpha_{RS} = \frac{RS - B}{\Gamma(1+\gamma^{-1})}$ $\alpha_{RA} = \frac{RA - B}{\Gamma(1+\gamma^{-1})}$

and we have $\frac{1}{\alpha^{\gamma}} = \frac{1}{\alpha_{RS}^{\gamma}} + \frac{1}{\alpha_{RA}^{\gamma}} = \frac{\alpha_{RS}^{\gamma} + \alpha_{RA}^{\gamma}}{\alpha_{RS}^{\gamma}\alpha_{RA}^{\gamma}}$

so $= 1 - \int_0^{\infty} \frac{\gamma}{\alpha_{RS}} \left(\frac{x}{\alpha_{RS}}\right)^{\gamma-1} e^{-(x/\alpha)^{\gamma}} dx$

these properties are used to transform the above

$\left(\frac{x}{\alpha_{RS}}\right)^{\gamma-1} = \left(\frac{x}{\alpha}\right)^{\gamma} \cdot \left(\frac{\alpha}{\alpha_{RS}}\right)^{\gamma-1}$ and $\frac{1}{\alpha_{RS}} = \frac{1}{\alpha} \cdot \left(\frac{\alpha}{\alpha_{RS}}\right)$ ← combine

$\frac{\gamma}{\alpha_{RS}} \left(\frac{x}{\alpha_{RS}}\right)^{\gamma-1} = \frac{\gamma}{\alpha} \left(\frac{x}{\alpha}\right)^{\gamma-1} \cdot \left(\frac{\alpha}{\alpha_{RS}}\right)^{\gamma}$

so $\text{Prob}(X > Y) = 1 - \frac{\alpha^{\gamma}}{\alpha_{RS}^{\gamma}} \int_0^{\infty} \frac{\gamma}{\alpha} \left(\frac{x}{\alpha}\right)^{\gamma-1} e^{-(x/\alpha)^{\gamma}} dx$

↑ pdf so must be 1 when evaluated

so $= 1 - \frac{\alpha^{\gamma}}{\alpha_{RS}^{\gamma}} = 1 - \frac{1}{\alpha_{RS}^{\gamma}} \cdot \frac{\alpha_{RS}^{\gamma}\alpha_{RA}^{\gamma}}{\alpha_{RS}^{\gamma} + \alpha_{RA}^{\gamma}}$

$= 1 - \frac{\alpha_{RA}^{\gamma}}{\alpha_{RS}^{\gamma} + \alpha_{RA}^{\gamma}} = \frac{\alpha_{RS}^{\gamma}}{\alpha_{RS}^{\gamma} + \alpha_{RA}^{\gamma}}$

subbing in what α_{RA} and α_{RS} mean

$P(X > Y) = \frac{(RS - B)^{\gamma}}{(RS - B)^{\gamma} + (RA - B)^{\gamma}}$

Figure 19: Pythagorean formula derivation (part 2)